STAT 6685 Final Project
Robust Rotary Bearing Fault Detection

Spencer Kenison[1]
Utah State University, Logan, UT 84321

I. Introduction

In mechanical engineering and manufacturing, avoiding unexpected failure in machinery helps to prevent system failure, costly downtime, and physical dangers. High usage and stress in rotating components such as bearing often make them one of the first parts of a system to fail. In the past, monitoring the condition and performance of rotating components was done by collecting vibration frequency data using accelerometers and then manually using signal processing techniques such as Fourier transforms to identify possible faults or issues.

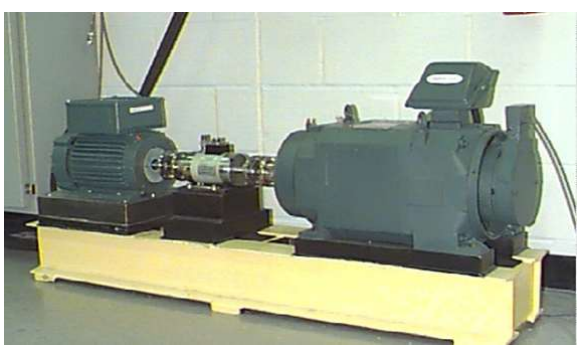
The purpose of the project is to create a deep learning model combining a CNN and LSTM to enable automated processing and diagnosis of with rotary equipment. High-frequency accelerometer data collected from rotating equipment is treated as sequential input and fed into a neural network. The neural network is trained to recognize the non-linear features of the sequential data that correspond with both normal function and various mechanical issues. This project includes designing, building, training, and evaluating a deep learning model capable of determining the current health of a rotary bearing under different loading conditions. The model is made more robust by utilizing a noise-augmented data set for training. A 1D Grad-CAM is implemented to visualize and better understand model activation.

II. Dataset and Preprocessing

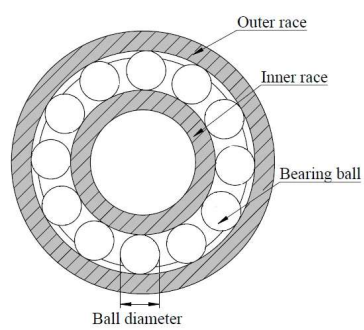
This section describes the dataset used to create the deep learning model, in addition to how the dataset is prepared for model training.

A. Data Source

This project is built around the Cases Western Reserve University (CWRU) bearing dataset [1]. This is a common data set used for models trained to diagnose mechanical faults. The data was collected using accelerometers placed on the drive end, fan end, and base of a motor housing as shown in Figure 1a. Continuous, sequential data from each accelerometer was collected under various motor loadings from 0 to 3 horsepower.



a) CWRU data collection setup. [1]



b) Diagram of key parts of a rotary bearing. [2]

Figure 1. Images explaining CWRU data collection setup.

Initially, data was collected from normal, functioning bearings. Single-point defects of various depths were then induced onto the ball, inner raceway, and outer raceway of the bearing using electro-discharge machining (EDM) as shown in Figure 1b. After each singular defect was created, accelerometer data was then collected in the same range of motor loading conditions. The data was collected as a set of 1-dimensional time series arrays sampled at both 12,000

[1] Graduate Student, USU Mechanical and Aerospace Engineering.

and 48,000 kHz. For this project, only data from the drive end and fan end accelerometers are used to train and test the deep learning model, creating a dual-channel model.

B. Segmentation

The data used for training is made up of long, high-frequency sensor recordings. The continuous raw accelerometer data are divided into smaller, fixed-size chunks using a sliding window approach. A common approach to this is creating sets of 1024 or 2048 data points for each sequence [3]. This transforms the few, long recorded sequential data sets into many discrete training data sets more suitable for deep learning model training. In this project, a sliding window size of 1024 with a stride of 1024 is used.

C. Data Preparation

Each segmented data set is normalized using z-score standardization, causing each segment to have the same relative mean and standard deviation. This regularization helps stabilize stochastic gradient descent (SGD), which is a key part of model training [4].

The raw data set is randomly split into 70% training, 15% validation, and 15% testing split prior to any model training. This is done to provide a large enough data set for accurate model training while still giving statistically significant data sets for model validation and final testing. As the split is applied, the data is stratified to ensure that every fault type, defect size, and loading condition are equally represented within the training, validation, and testing data sets. This stratification is important to avoid the possibility of one fault type being overrepresented in the training, validation, or testing data sets [5].

With the CWRU bearing data used, 4640 separate 1024 x 2 segments are extracted from the raw time-series data. This includes 472, 1319, 1425, and 1424 segments from the normal, inner ring, outer ring, and ball defect data sets respectively. Once the data has been split, 3248 segments are devoted to model training, 696 to model validation, and 696 to model testing. An example of these raw data series is shown in Figure 1.

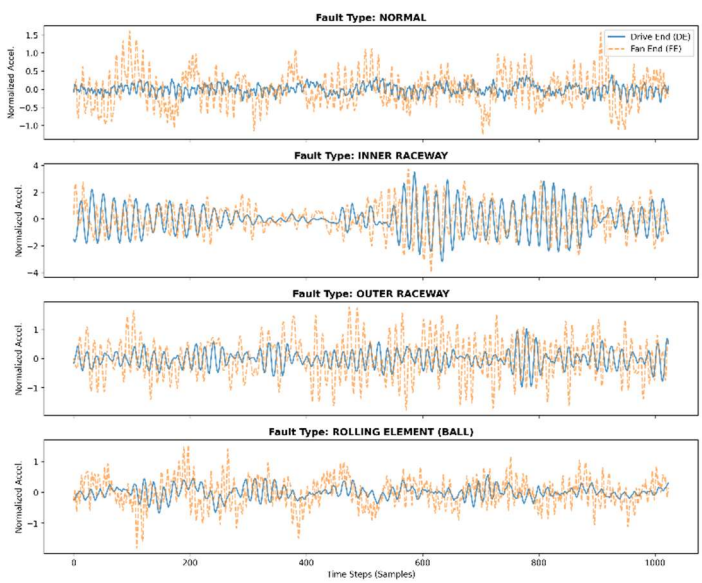


Figure 1: Plotted examples of 1024 x 2 segments of each data type.

As shown in Figure 1 and as mentioned previously, each chunk of data fed into the deep learning model includes 1024 pieces of data on 2 channels. Visually identifying key features indicative of specific failure modes would be difficult and require significant time and attention, something that is not in excess in any environment where rotary machines are used to reduce load on individuals. Creating a deep-learning model to automatically detect and identify system failures enhances system autonomy.

III. Model Architecture

This section outlines the architecture of the deep learning model used to recognize various bearing errors using the CWRU dataset. The model is built using PyTorch implemented via a Jupyter notebook for simplicity and compatibility.

The deep learning model architecture for this project is made up of a combination of a 1D convolutional neural network (CNN) and Long Short-Term Memory (LSTM). The combined model architecture is needed to interpret a complex set of data that changes over time. The CNN highlights the features from each chunk of data with the LSTM examining the feature occurrences over time to find meaningful patterns within the data [6] [7].

A. Feature Extraction: 1D Convolutional Neural Network

The initial 1-dimensional CNN is used to extract local spatial features, like high-frequency spikes or periodic impacts, from the raw temporal 1024 x 2 data sets. The input layer of the CNN accepts a batch of 1024 x 2 sets of data.

The first convolution block consists of 16 filters, meaning that it is attempting to identify 16 different types of patterns simultaneously. This convolution block has a kernel size of 64 to capture larger temporal features along with a stride of 2. Non-linear ReLU activation is used to help the model identify complex patterns by ignoring neurons that are not finding useful information [8]. To avoid excessive model size and computational load, MaxPool1d with a window size of 2 is used to keep important data while halving the size of the model.

The second convolution block consists of 32 filters, a kernel size of 32, and a stride of 2. It also uses ReLU activation and MaxPoold with the same configuration as the first convolution block. Sequence Learning: Long Short-Term Memory

B. Temporal Patterns: Long-Term Short Memory

After the CNN has down sampled the raw frequency data into a high-level set of features, the LSTM identifies patterns that occur over long spans of time. Initially, the data is reshaped from a batch, channel, sequence configuration to batch, sequence, channel configuration to meet the recurrent neural networks (RNN) input requirements.

The RNN layer is made up of a single-layer LSTM with 64 hidden units, which are the different internal variables that makeup the "memory" of the model. It processes the 52-step, 32-channel sequence produced by the CNN to learn temporal dependencies and long-term relationships between the extracted features.

The only output used by the model from the LSTM is its final hidden state. This could be thought of as the summary of the "thoughts" of the LSTM as it processes the data from the CNN.

C. Classification: Multi-Layer Perceptron

The 64-item summary from the LTSM is used as the input into a multi-layer perceptron (MLP) to determine if the bearing is functioning normally or experiencing some form of failure. Like the CNN, the MLP utilizes non-linear ReLU neuron activation throughout. The MLP begins the categorization process by randomly dropping 50% of the data, regularizing the data to prevent overfitting to specific neurons during training. The first linear layer of the MLP reduces the 64 hidden units from the LSTM to 32 units, with the "linear' term signifying that each neuron at one layer is connected to all the neurons at the next layer.

The second linear layer of the MLP maps the 32 features to four logit. Each of the logits represents the raw scores of one of the four classes: normal, inner raceway defect, outer raceway defect, or ball defect. The highest score becomes the model's classification of bearing function for a particular chunk of data.

IV. Model Training

The basic architecture of a deep learning model is not valuable unless it has been trained to properly respond to the desired dataset(s). In this case, a supervised, backpropagation learning workflow is used to train the deep learning model to identify fault types in the CWRU bearing dataset.

A. Optimization

Training of a deep learning model such as this happens through minimizing a loss function that attempts to describe how far the deep learning model is from identifying or representing truth. In this deep learning model, a cross-entropy loss function is used based on the output from a vector of the four logits corresponding with each class. The cross-entropy loss function applies a SoftMax function to turn the logit scores and calculates the distance from the prediction and the actual label.

For a single sample where s_p represents the score for the correct class and s_j represents the individual logits that make up the output vector of the deep learning model, the loss function appears as follows:

$$Loss = -\log\left(\frac{exp(s_p)}{\sum_j exp(s_j)}\right)$$

The deep learning model uses Stochastic Gradient Descent (SGD) with momentum to modify the weights and biases of the model to minimize the loss. The gradient of the loss with respect to every weight and biases within the CNN and LSTM is calculated using a backward step. The model then modifies the weights and biases of the CNN and LSTM based on the gradient of the loss in the direction that minimizes the loss.

This modification based on SGD is complimented by using momentum, which modifies the weights and biases based on the previous update step in addition to the current update step. This helps the optimizer quickly move past local minima and maxima within the loss landscape of the time-series data. In this case, a momentum value of 0.9 is used to find a balance between speed and stability in model training [9].

The learning rate value specified in the model determines the step size the optimizer takes when updating the model weights. The learning rate acts as a multiplier, with the new weight being the difference between the old weight and product of the learning rate and calculated gradient. A standard learning rate of 0.01 is used in this deep learning model.

B. Initialization and Regularization

The weight values within the model are initialized prior to model training using He initialization. In He initialization, weights are initialized from a normal distribution of $\sigma^2 = \frac{2}{n_{in}}$, where n_{in} is the number of input units in a layer. This helps to avoid unstable forward passes in the initial portion of model training [10]. Because the model uses ReLU activation functions, excessively small initial weights could cause vanishing gradients, while excessively large weights could risk gradient instability.

To prevent excessively large weights that could cause the model to become overly sensitive to small changes in the input signal, L2 weight decay is used. L2 weight decay adds a penalty to the calculated loss function based on the square of the weight value (i.e. $Loss = Loss_{calculated} + \lambda||w||^2$ where w represents the weight value). In this case, L2 weight decay of 1e-04 is used to avoid excessively large weights.

C. Training Loop

Each epoch, or full pass through the data, during training follows the same sequence. Initially, the gradients are cleared from the previous batch to avoid accumulation. Then, a forward pass is performed with the 1024-sample window fed through the network to get a prediction. The loss calculation is performed to determine how the prediction compares to the actual sample label.

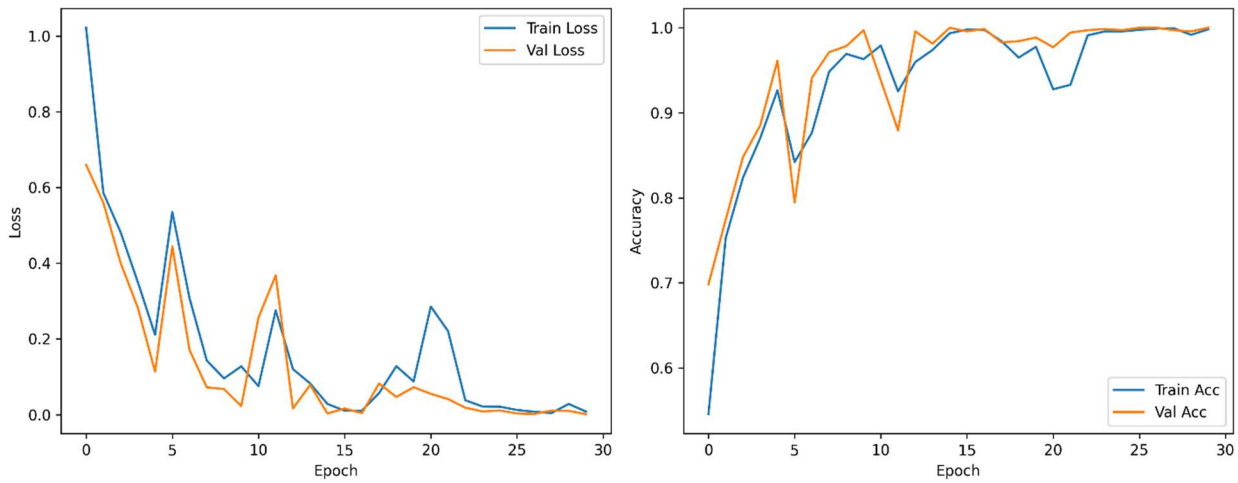
A backward pass, or backpropagation, is then performed by sending the error backwards through the MLP, LSTM, and CNN. The weights are then updated based on the calculated gradients and specified learning rate. The training loop is repeated multiple times to achieve the desired level of model accuracy.

V. Results and Evaluation

This section details the training of the deep learning model and the initial performance of the model.

A. Training Results

The model was trained using 30 epochs of SGD with a learning rate of 0.01, momentum of 0.8, and $1x10^{-4}$ weight decay via L2 regularization, as discussed previously. The performance of the model based on the training and validation data subsets are shown in Figure 2.



a) Loss over Epochs b) Accuracy over Epochs

Figure 2. Loss and accuracy during model training.

The model appears to rapidly converge within the first 5 epochs, with an accuracy of above 80% relative to training and validation data. As the model continues to train, some small jumps are seen in the loss and accuracy data, with the model converging further after each. This is expected when using a training method like SGD on vibrational data using momentum, where the model could slide past local minimum temporarily. Both the training and validation curves appear to converge at 25+ epochs, with the loss approaching zero and accuracy approaching 100%.

After the model appeared to converge at its maximum possible accuracy, the training process is stopped to prevent the model from overfitting and learning the noise within the training data.

B. Model Evaluation

After testing, the model is evaluated using the testing data allocated previously. This independent evaluation using data not touched by the model during training better evaluates overfitting and generalization of the trained model, compared to model performance with respect to the training and validation data.

The trained model demonstrates a final test accuracy of 99.14% with testing data. Table 1 displays the classification report of the model with respect to the testing data subset.

	Precision	Recall	F1-Score	Support
Normal	1.00	1.00	1.00	71
Inner Raceway Defect	0.99	0.97	0.98	198
Outer Raceway Defect	1.00	1.00	1.00	214
Ball Defect	0.98	1.00	0.99	213
Accuracy			0.99	696

Table 1. Classification report of the model using testing data subset.

The model shows near-perfect classification across all four categories. Precision and recall for the normal data and outer raceway defect were identified perfectly, with no faults positives or missed data within those classes. The specific confusion cases by the model between the inner raceway defect and ball defect can be seen in the confusion matrix of testing shown in Figure 3.

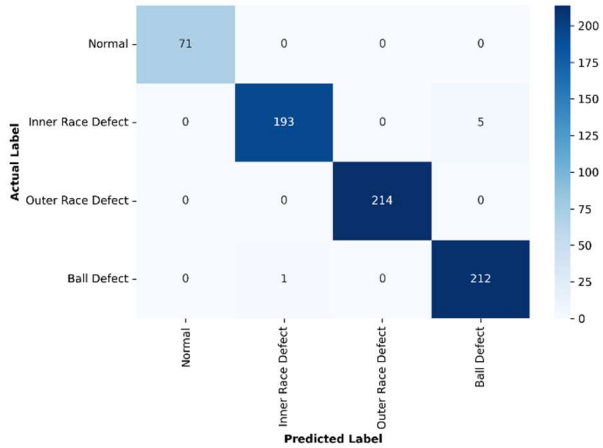


Figure 3. Confusion matrix of trained model on testing data subset.

The confusion matrix shows that 5 inner raceway defects are misidentified as ball defects, with 1 ball defect being misidentified as an inner raceway defect. Confusion of some kind is to be expected in a system such as this, where inner raceway and rolling element are both moving components. Accordingly, defects in those two systems could possibly have overlapping vibrational behavior in the frequency domain, contrasting to the outer raceway which is stationary.

VI. Model Robustness

Although the trained model responded well to testing data, it must be recognized that most real-world environments where bearings are used and monitored, such as manufacturing floors, are not as acoustically clean as the setup used to collect the CWRU data. To account for this, the model is re-trained using a comparatively noisy dataset. The robust, noise-trained model performance is then compared to the original, clean-trained model when tested using noisy data.

A. Noise Injection

To simulate acoustically noisy reality of most industrial environments, the CWRU data is augmented using Additive White Gaussian Noise (AWGN) [11]. During each training epoch, gaussian noise is injected dynamically into the training data. The signal-to-noise ratio (SNR) is randomly sampled from a uniform distribution between -5 dB, or extreme noise, and 30 dB, a clean signal. This process forces the 1D-CNN layers within the model to ignore the transient background signals within the noisy data and focus on the basic periodic patterns that indicate each fault type.

B. Robust Model Training

The model was trained using 40 epochs of SGD with a learning rate of 0.01, momentum of 0.8, and $1\text{x}10^{-4}$ weight decay via L2 regularization. All hyperparameters matched the training of the clean-data trained model, besides an increase in epoch count to account for the slower convergence rate. The performance of the robust, noise-trained model based on the training and validation data subsets are shown in Figure 4.

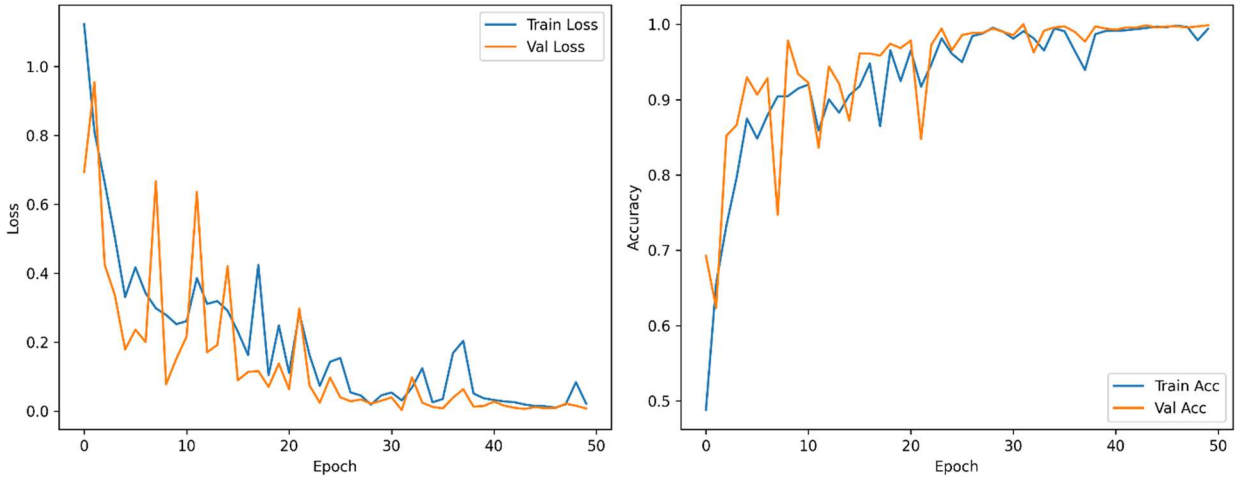


Figure 4. Loss and accuracy during noise injection model training

The 50 epochs required to train the model based on noise-injected data contrast the 30 epochs required for the model trained on clean data. This is sensible given that the model needed to overcome the additional complexity of overcoming noise within the data.

Although somewhat steady convergence is observable from a high level, sizable intermediate temporary increases in loss or decreases in accuracy occur from epoch to epoch. This can be attributed to the randomly assigned AWGN that is applied at each epoch. At epochs where a relatively higher SNR is applied, a temporary increase in loss and decrease in accuracy is expected.

C. Comparative Results

Comparison of the models trained without and with noise augmentation can help determine the effectiveness of AWGN data augmentation during training. Performance of each model with respect to noise is determined by verifying model performance using the testing data with a range of set levels of AWGN applied. The accuracy of each model with respect to the SNR is shown in Figure 5.

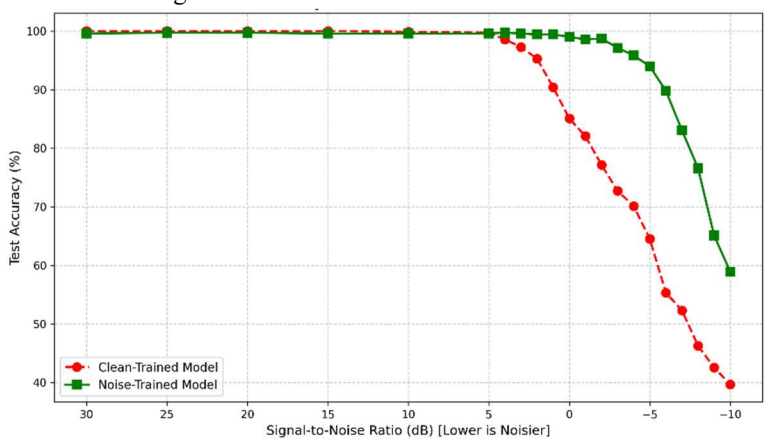


Figure 5. Robustness comparison of clean versus noise-trained data.

At relatively high SNR values ranging from 30 dB to 5 dB, both models perform well, maintaining an accuracy above 99.5% This indicates that the additional noise during training of the robust model did not significantly impact the robust model's ability to classify cleaner data.

However, as the SNR drops below 5 dB, the accuracy of the model trained without noise suddenly drops off, reaching 85.1% accuracy at 0 dB SNR where the noise power equals signal power. The robust model appears more resilient, not displaying the same sudden drop off in accuracy at 5 dB and maintaining 99.0% accuracy at 0 dB SNR. As noise power increases beyond signal power at less than 0 dB, the robust model continues to outperform the clean-data trained model, maintaining 94.0% accuracy as an SNR of -5 dB, where the noise power is more than 3 times more than signal power.

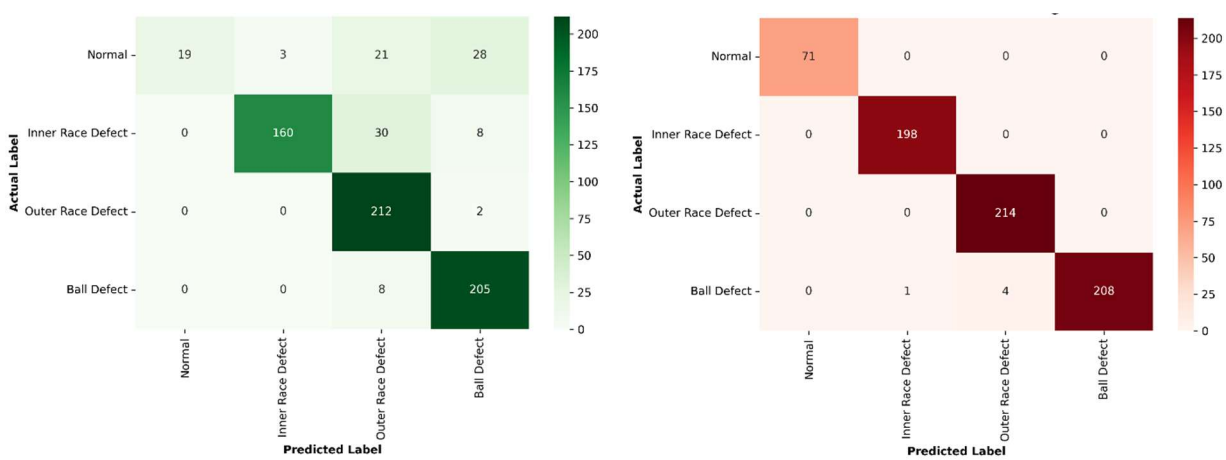
The improved performance of the robust, noise-trained model can be further validated by comparing the classification reports of the models shown in Table 2 and the confusion matrixes shown in Figure 6. These comparisons were done with 0 dB SNR applied to the testing data where the noise and signal power are equal.

	Clean-Trained Model				Noise-Trained Model			
	Precision	Recall	F1-Score	Support	Precision	Recall	F1-Score	Support
Normal	1.00	0.27	0.42	71	1.00	1.00	1.00	71
Inner Raceway Defect	0.98	0.81	0.89	198	0.99	1.00	1.00	198
Outer Raceway Defect	0.78	0.99	0.87	214	0.98	1.00	0.99	214
Ball Defect	0.84	0.96	0.90	213	1.00	0.98	0.99	213
Accuracy			0.86	696			0.99	696

Table 2. Classification reports comparing clean-trained and noise-trained models at 0 dB SNR.

The classification report of the clean-data trained model clearly indicates that when noisy data is fed into the model, it fails to correctly classify normal bearings, showing a normal recall of 0.27 with 73% of normal bearings being misclassified as defective in some way. In application, this would cause a high number of false alarms in an industrial or manufacturing environment. Additionally, there appears to be significant levels of confusion between each defect type.

The noise-trained, robust model does not share the same false-alarm or confusion issues as the clean-trained model. In this case, the robust model can perfectly identify normal bearing with a precision and recall of 1.00. Across all defect categories, the robust model shows a recall and precision of 0.98 or higher. This indicates that the robust model successfully learned to identify the important features and filter out the noise during training.



a) Model trained without noise b) Model trained with noise injected

Figure 6. Confusion matrix comparison at 0 dB SNR for models trained without and with noise injection.

The confusion matrices verify the results summarized in the classification reports. The clean-trained model misclassifies 52 out of 71 of the healthy samples, mostly as outer raceway or ball defects, suggesting that the injected noise is mistaken for the high-frequency features of rotating element defects. The robust model only misclassified 5 ball defects as some sort of raceway defect while correctly identifying all other 691 samples of 0 dB SNR data.

These results show the significant strengthening of a model's performance when clean training data is augmented with AWGN.

VII. Model Visualization

To more deeply understand model activation and the CNN-LSTM architecture, a 1D gradient-weighted Class Activation Map (1D Grad-CAM) visualization is implemented.

A. 1D Grad-CAM

A 1D Grad-CAM can identify which portions of a sample contribute most to the final sample classification [12]. This is done by attaching forward and backward "hooks" to the final convolutional layer within the model, copying the feature maps and the corresponding gradients. Global Average Pooling is then performed on the gradients to calculate weights, which in turn are used to calculate a weighted sum of model activations. ReLU activation is applied to focus only on positive activations [8]. The model's activation map is then upsampled to the original sample length of 1024 using linear interpolation.

B. Visualizing Features

The 1D Grad-CAM heatmap visualization is overlayed on the accelerometer data to visually correlate model activations with accelerometer data, as shown on a random set of samples from each defect type in Figure 7. This visualization is created using clean data fed into the robust, noise-trained model to not include confusing noise in the visualization. The model activation is min-max normalized to simplify comparison.

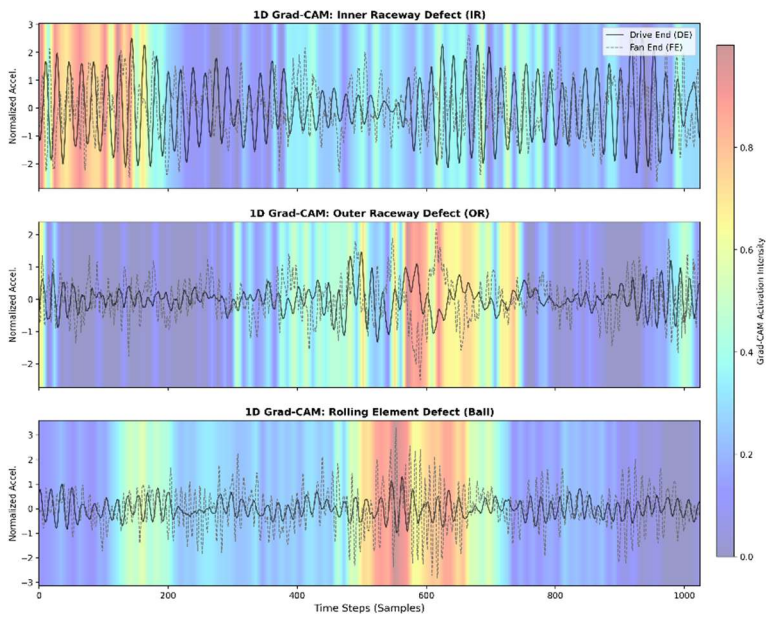


Figure 7. Visualized 1D Grad-CAM showing activation of robust model for each defect type.

Each defect has differing accelerometer trends corresponding with model activation. The inner raceway defect's model activation appears to be trigged by high-amplitude, sudden spike observed in the accelerometers on both the drive and fan ends of the testing setup. The rolling element defect appears to activate the model with high-amplitude, high-frequency spikes specifically on the fan end accelerometer. The activation of the model corresponding to the outer raceway bearing defect is not immediately as apparent, although a series of higher amplitude rises and falls in both fan and drive end accelerometer readings are visible at model activation.

These visualizations confirm that the trained model successfully combines trends from both the drive end and fan end accelerometer channels to properly recognize each defect. Further study of 1D Grad-CAM activation maps could enable users to better visually recognize defects as they occur to validate model-driven defect classifications.

VIII. Conclusion

This project successfully creates a deep learning CNN-LSTM model to detect faults within rotary bearings. The model uses a CNN to extract features and an LSTM to learn sequences indicative of specific types of faults. The trained model has 99.1% accuracy, correctly identifying faults with clean testing data. The high accuracy of the model shows that a combined CNN-LSTM model is an effective signal processing tool to automatically identify faults.

Testing the model trained with clean data using Additive White Gaussian Noise (AWGN) augmented test data indicated that the model is not resilient to environmental noise that would likely exist in real-world applications, showing a drop to 86% accuracy at 0 dB signal-to-noise ration (SNR). Training a model on AWGN-augmented data resulted in a much more noise-resilient model, achieving 99.0% accuracy on testing data with 0 dB SNR. This shows that a model trained with noise present can learn to filter out background noise and focus on periodic patterns in accelerometer data indicative of faults. A 1D Grad-CAM visualization shows how the model was activated by particular features in the data that triggered specific class designations.

This project could be expanded by performing training and testing using data collected at various speeds, given that most rotary systems operate at a range of speeds in application. Additionally, the trained CNN-LSTM network could be deployed to actual systems to validate the model's effectiveness and enable real-time monitoring of rotary system status.

References

[1] Case Western Reserve University, "CWRU Bearing Dataset," [Online]. Available: https://www.kaggle.com/datasets/brjapon/cwru-bearing-datasets/data. [Accessed 28 February 2026].

[2] G. Pagiatakis, L. Dritsas, G. Chatzarakis, G. Todorov and B. Stoev, "Introducing concepts and methodologies of fault detection into electrical engineering education: The induction machine example," in *2017 IEEE Global Engineering Education Conference (EDUCON)*, Athens, 2017.

[3] M. Wang, L. Zhang, H. Yu, S. Chen, X. Zhang, Y. Zhang and D. Gao, "A deep learning network based on CNN and sliding window LSTM for spike sorting," *Computers in Biology and Medicine,* vol. 159, 2023.

[4] J. Yun and H. Kim, *ZNorm: Z-Score Gradient Normalization for Deep Neural Networks,* Stony Brook, New York: arXiv, 2024.

[5] S. J. Jami, T. A. Li and J. L. Mueller, *Stratify or Die: Rethinking Data Splits in Image Segmentation,* Bayreuth: arXiv, 2025.

[6] S. Indolia, A. K. Goswami, S. P. Mishra and P. Aopa, "Conceptual Understanding of Convolutional Neural Network- A Deep Learning Approach," *Procedia Computer Science,* vol. 132, pp. 679-688, 2018.

[7] M. Krichen and A. Mihoub, "Long Short-Term Memory Networks: A Comprehensive Survey," *AI,* vol. 6, no. 9, 2025.

[8] H. Ide and T. Kurita, "Improvement of learning for CNN with ReLU activation by sparse regularization," in *International Joint Conference on Neural Networks (IJCNN)*, Anchorage, 2017.

[9] T. Lee, G. Gasswint and E. Henning, "Momentum," in *Cornell University Computational Optimization Open Textbook*, Ithaca, New York: Cornell University, 2021.

[10] L. Datta, "A Survey on Activation Functions and their relation with Xavier and He Normal Inititialization," *arXiv,* vol. abs/2004.06632, 2020.

[11] M. R. Ahmed, S. Islam, K. M. Islam and S. Shatabda, *An Ensemble 1D-CNN-LSTM-GRU Model with Data Augmentation for Speech Emotion Recognition,* Bangladesh: ArXiv, 2021.

[12] G.-y. Shi, H.-P. Wu and S.-h. Luo, "1D Gradient-Weighted Class Activation Mapping, Visualizing Decision Process of Convolutional Neural Network-Based Models in Spectroscopy Analysis," *Analytical Chemistry,* vol. 95, no. 26, p. 9959–9966, 2023.